

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

CS 498: Memory Optimization

Fan Lai

The Grainger College of Engineering

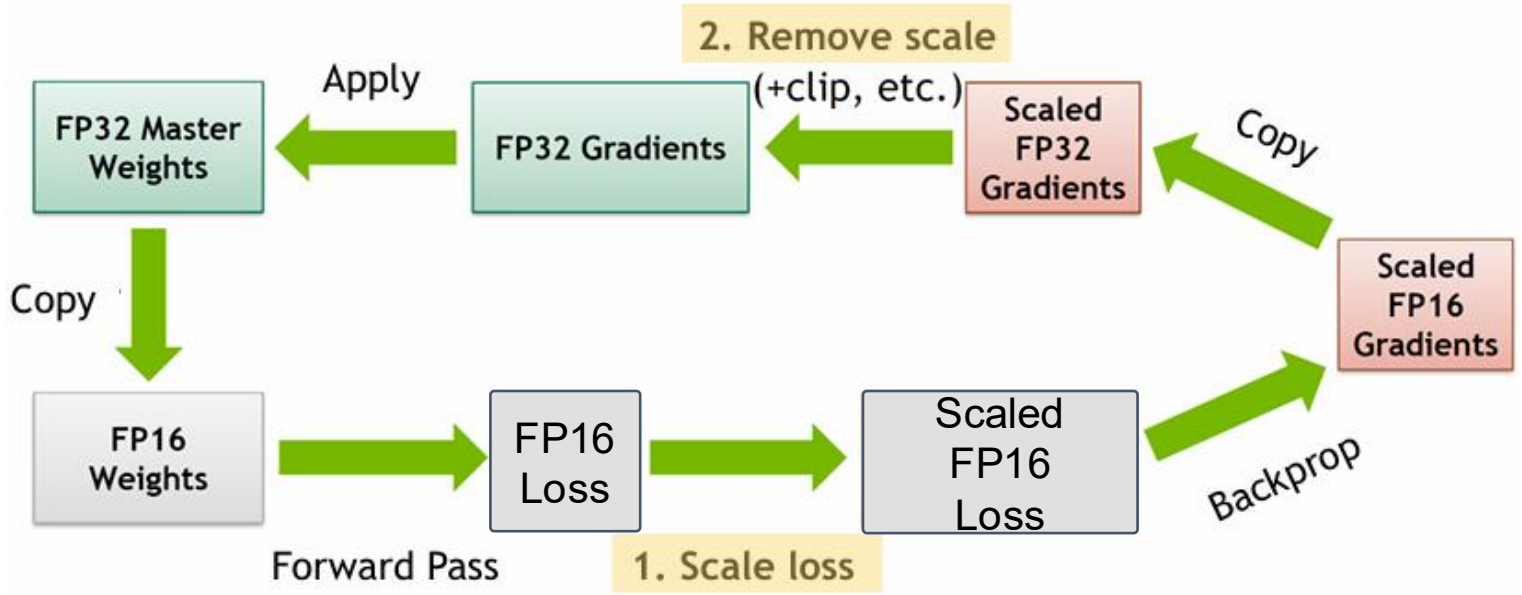
Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, and many colleagues

Assignment 1

- **A1 proposal due *this Wednesday*~**
- **(Remaining) Proposal feedback session**
  - (Optional) Stop by during office hours

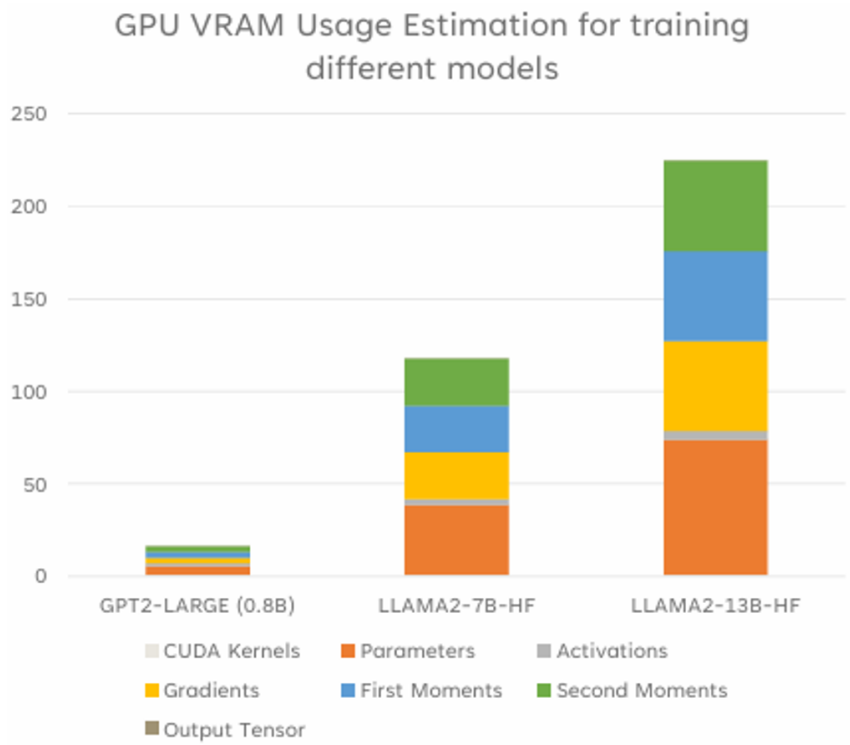
Recall: Mixed Precision Training

- **Mixed precision improves efficiency but may hurt quality** (FP16 vs. INT8)
- **Automatic Scaling at runtime may help quality**
  - E.g., scaling the loss to prevent underflow



Today: Memory Optimization

- **Task:** Reduce the memory consumption to train a DL model on single device
- **Main challenge:** Cost to store intermediate results and gradients

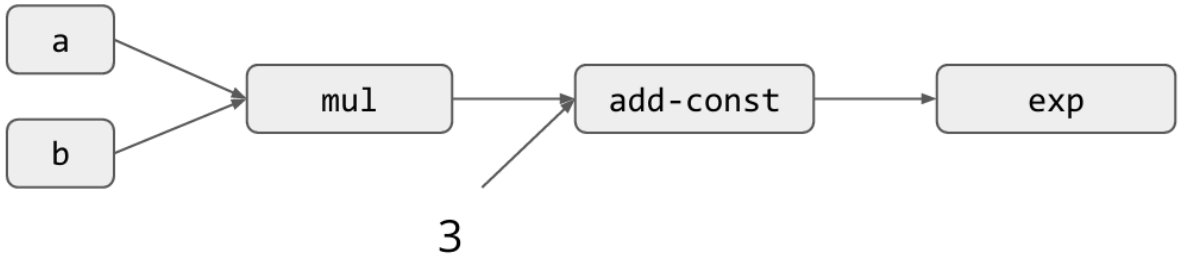


Today: Memory Optimization

- **Task:** Reduce the memory consumption to train a DL model on single device
- **Main challenge:** Cost to store intermediate results and gradients
- **Key Ideas:**
  - Computation *graph* memory optimization (In-place Storing, Memory Sharing)
  - Save memory by re-computing (Activation checkpointing)

Build an Executor for a Given Graph

Computational Graph for `exp(a * b + 3)`

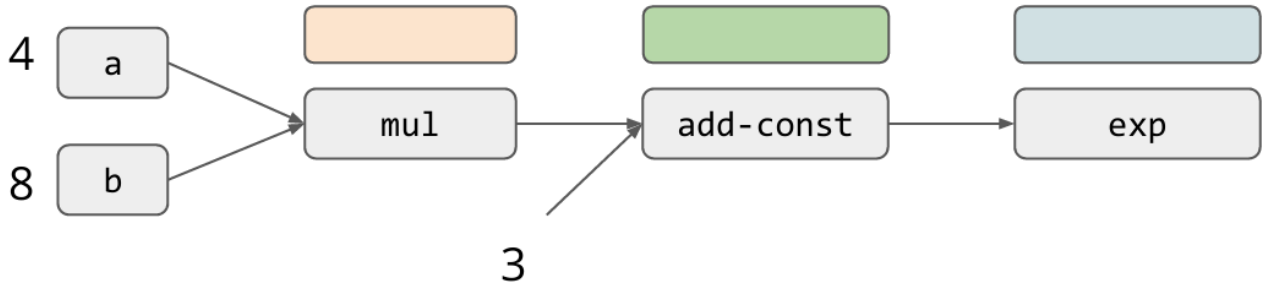


Build an Executor for a Given Graph

1. **Allocate** temp memory for intermediate computation

Computational Graph for `exp(a * b + 3)`

Same color represent same piece of memory

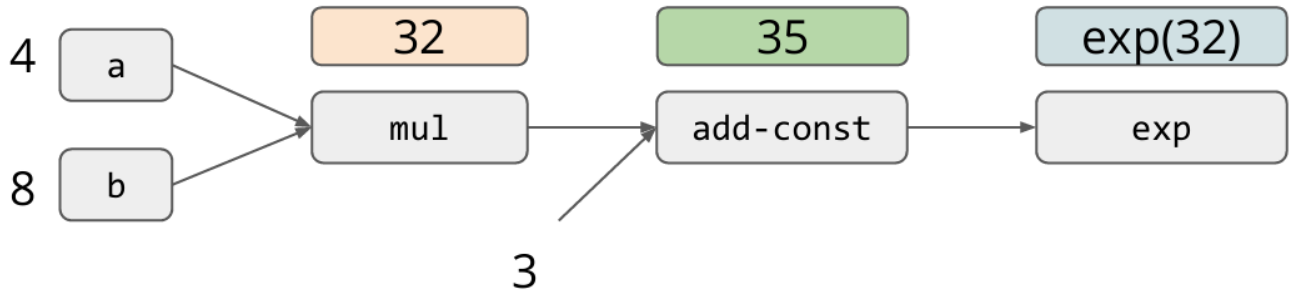


Build an Executor for a Given Graph

1. **Allocate** temp memory for intermediate computation
2. **Traverse and execute** the graph by topo order.

Temporary space linear to number of ops

Computational Graph for `exp(a * b + 3)`



Memory Management: Build an Executor for a Given Graph

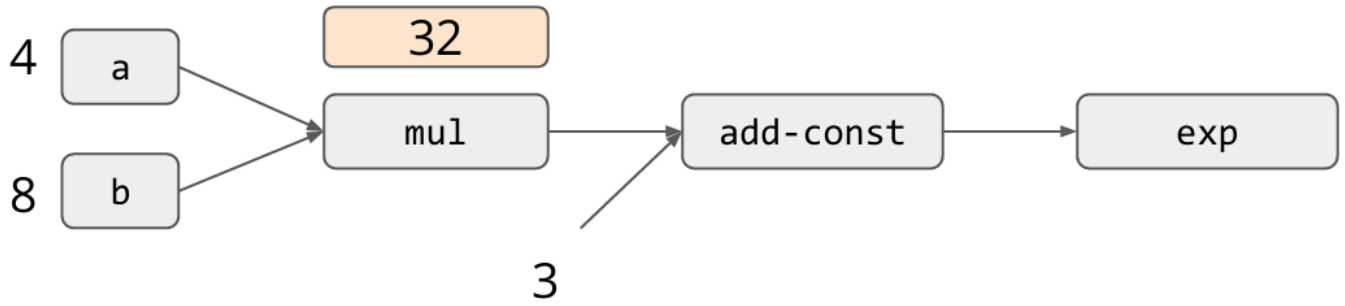
Dynamic Memory Allocation

Static Memory Allocation

Dynamic Memory Allocation

1. **Allocate** when needed
2. **Recycle** when a memory is not needed.

Memory Pool

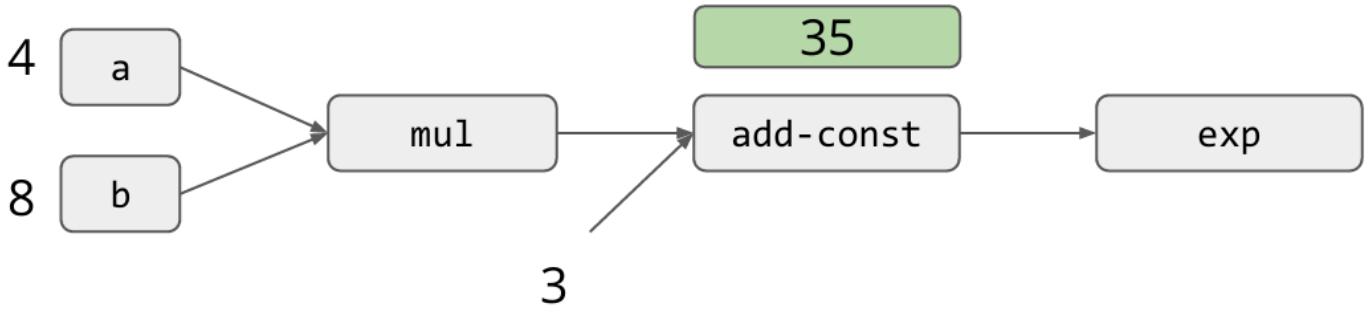


Dynamic Memory Allocation

1. **Allocate** when needed
2. **Recycle** when a memory is not needed.

Memory Pool

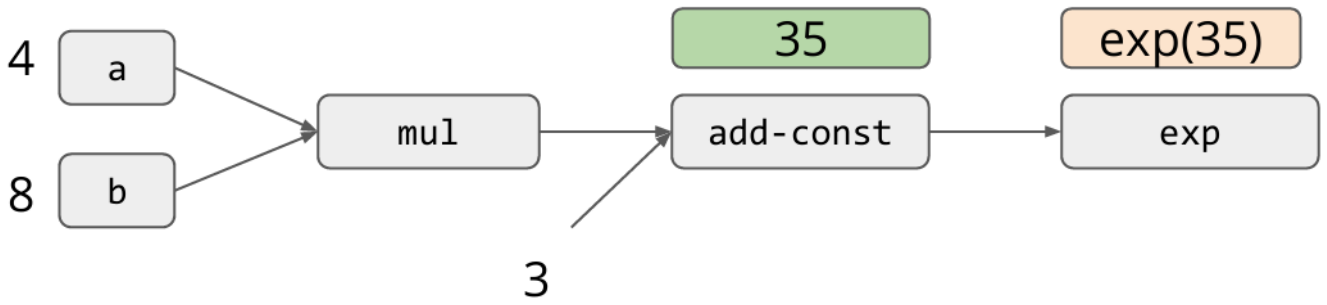




Dynamic Memory Allocation

1. **Allocate** when needed
2. **Recycle** when a memory is not needed.

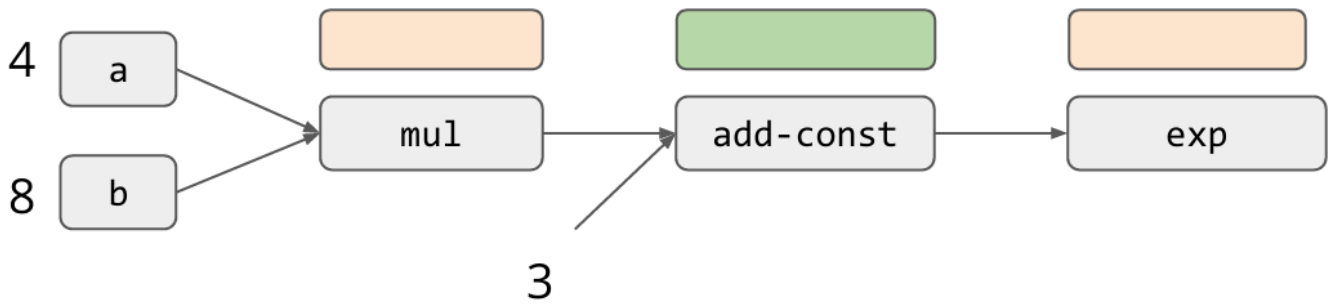
Memory Pool



Static Memory Planning

1. Plan for reuse **ahead of time**
2. Analog: register allocation algorithm in compiler

Same color represent same piece of memory



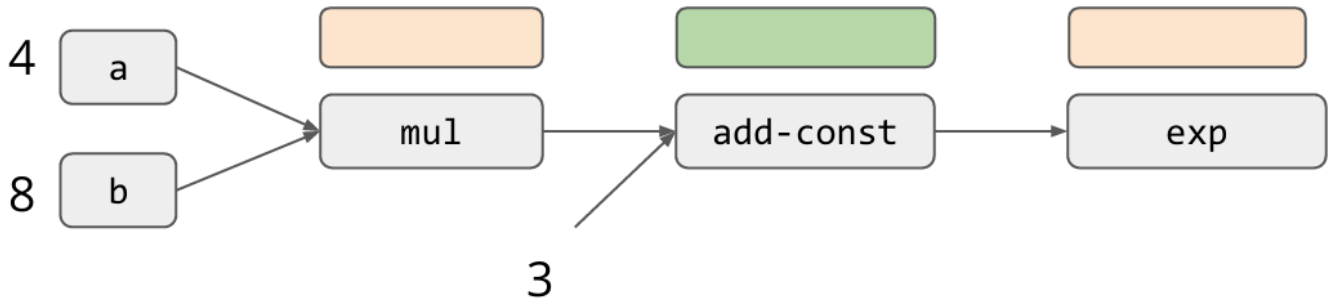
Static Memory Planning

1. Plan for reuse **ahead of time**
2. Analog: register allocation algorithm in compiler

TensorFlow 1.x, PyTorch 2.0 (static graph mode)

Question: What are pros and cons between static and dynamic allocation?

Same color represent same piece of memory



Memory Management

Dynamic Memory Allocation

- Can be done at runtime

Static Memory Allocation

- Determined at compile time

Memory Management

Dynamic Memory Allocation

- Can be done at runtime
- Flexible, more efficient use of memory

Static Memory Allocation

- Determined at compile time
- No runtime overhead

Memory Management

Dynamic Memory Allocation

- Can be done at runtime
- Flexible, more efficient use of memory
- Potential for fragmentation

Static Memory Allocation

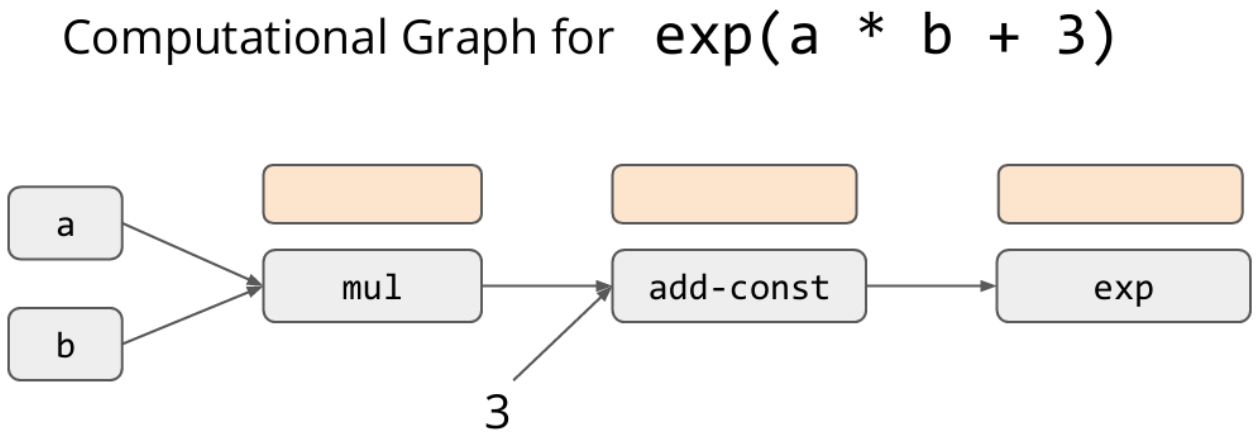
- Determined at compile time
- No runtime overhead
- Inflexible, can lead to wasted memory

Common Patterns of Memory Planning

- **Inplace** store the result in the input
- **Normal Sharing** reuse memory that are no longer needed.

Inplace Optimization

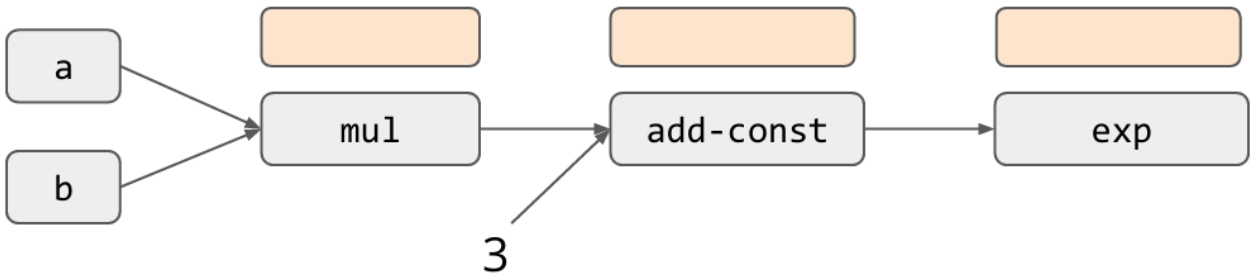
- Operations that modify data directly in the memory location where it is stored
- Examples: In-place activation functions, in-place BatchNorm
- Benefits:
  - Reduce memory footprint
  - Improve computation efficiency



Inplace Optimization

- Operations that modify data directly in the memory location where it is stored
- Examples: In-place activation functions, in-place BatchNorm
- Benefits:
  - Reduce memory footprint
  - Improve computation efficiency

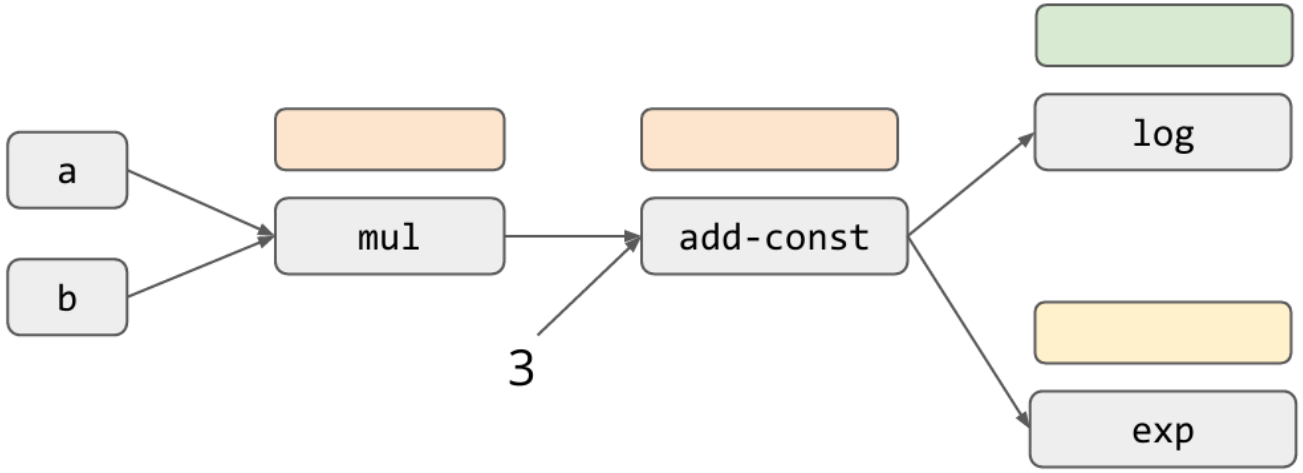
Computational Graph for `exp(a * b + 3)`



Question: What operation cannot be done in-place?

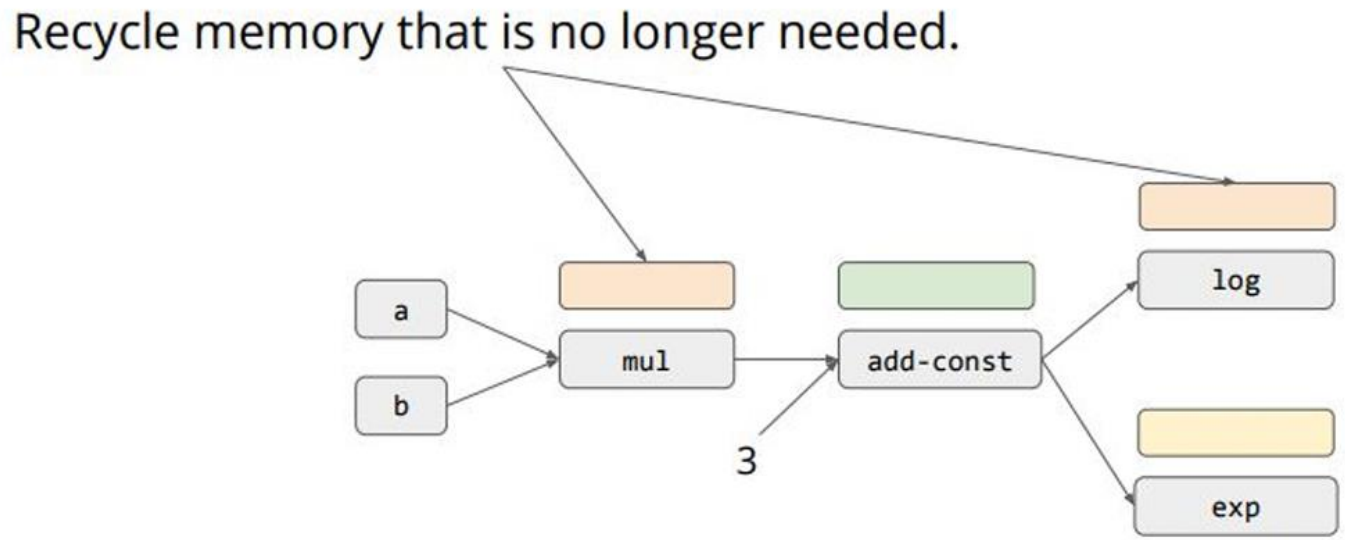
Inplace Optimization

- We can only do in-place computation if the result op is the only consumer of the current value (i.e., no other operator depends on the current value)



Memory Optimization: Memory Sharing

- Memory used by intermediate results that are no longer needed can be recycled and used in another node;
- Shared memory between the nodes whose lifetime do not overlap

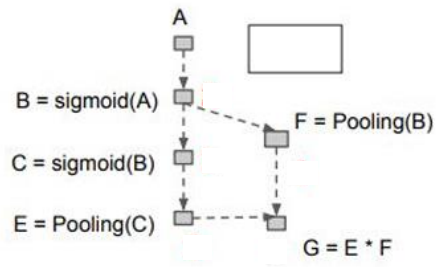


Memory Allocation/Planning Algorithm

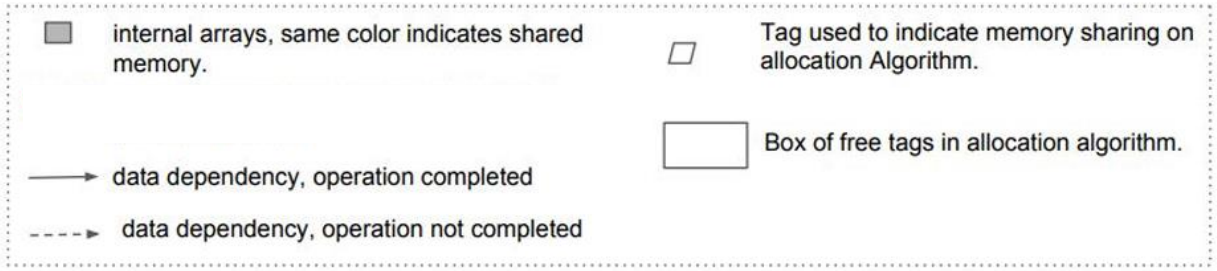
- Static Memory Planning: plan for reuse ahead of time

Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time

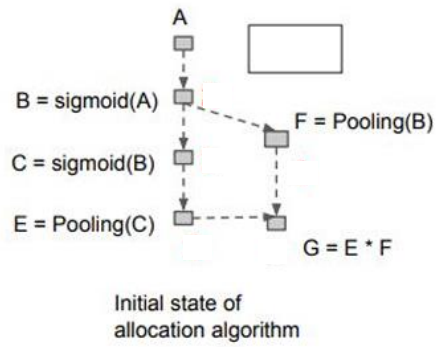


Initial state of allocation algorithm

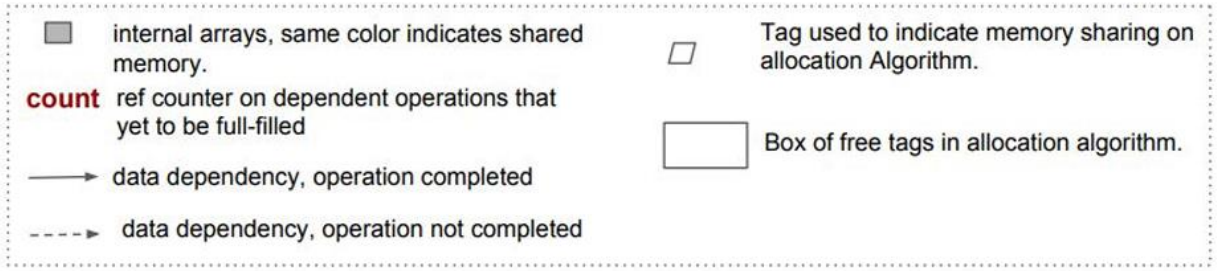


Memory Allocation/Planning Algorithm: Liveness Analysis

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused

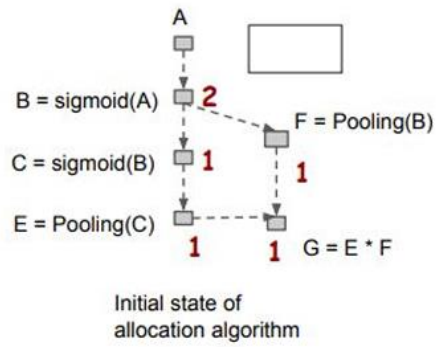


Initial state of allocation algorithm

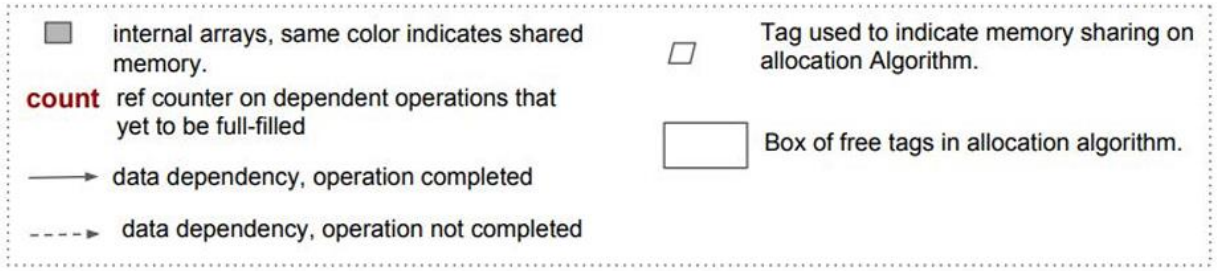


Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused

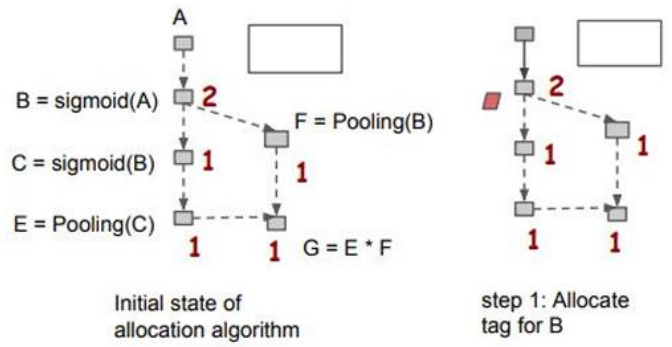


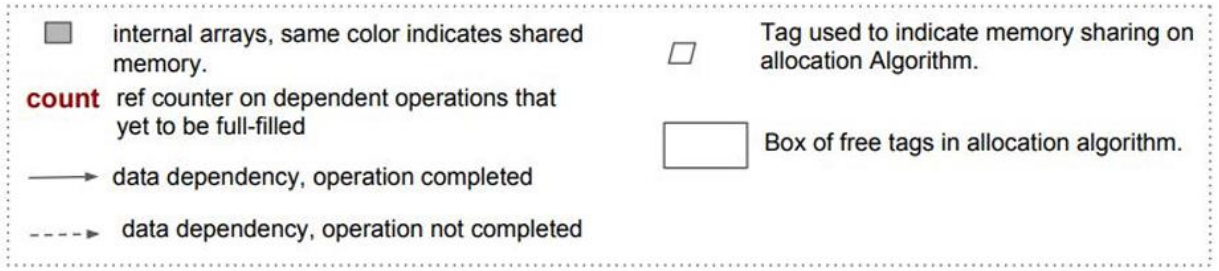
Initial state of allocation algorithm



Memory Allocation/Planning Algorithm

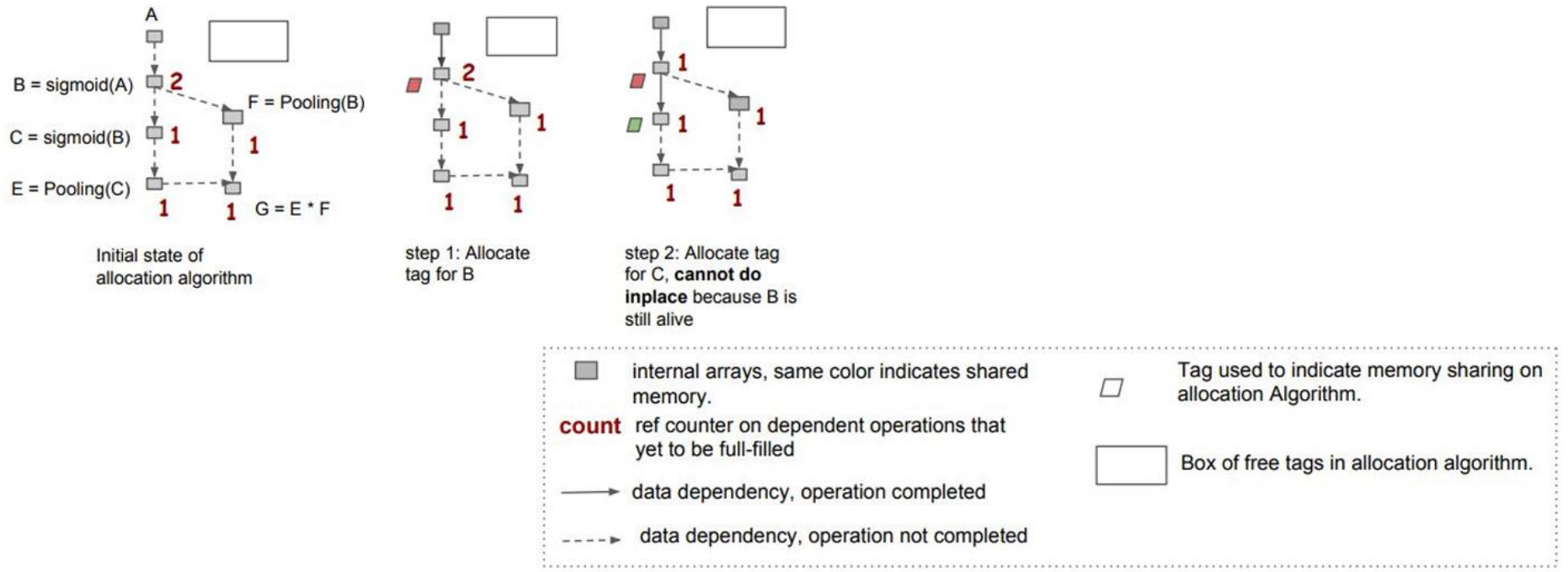
- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused





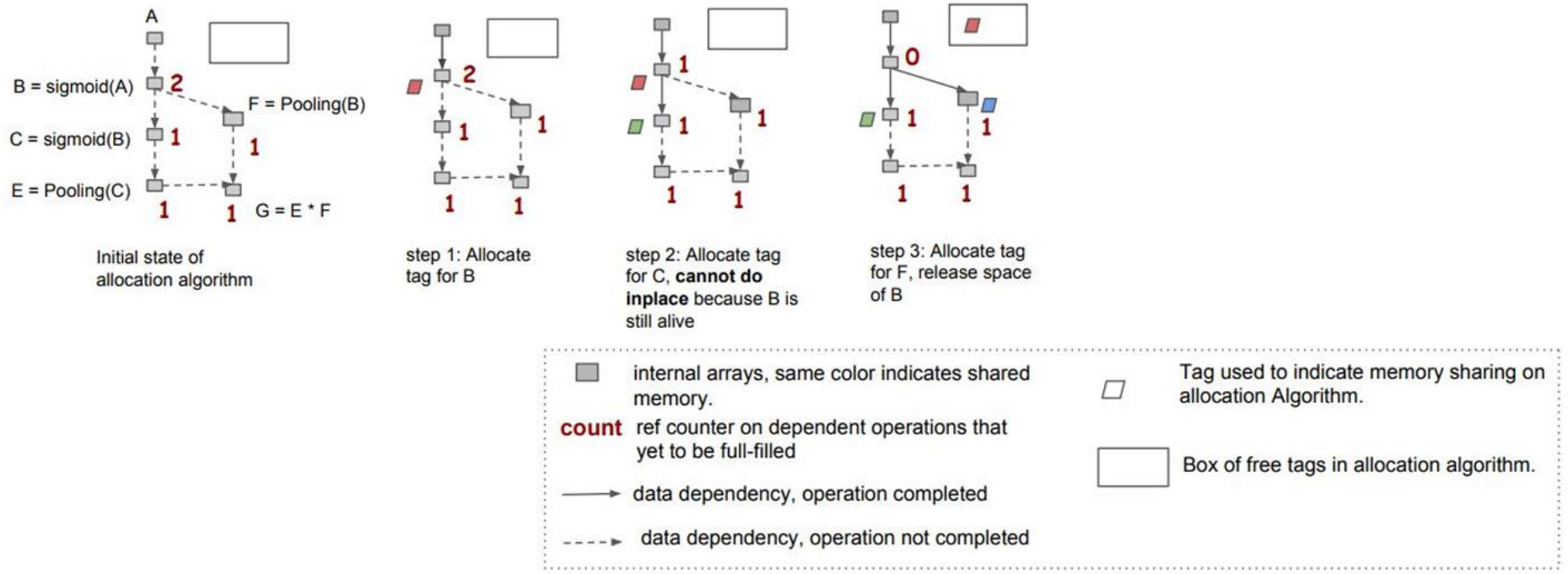
Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused



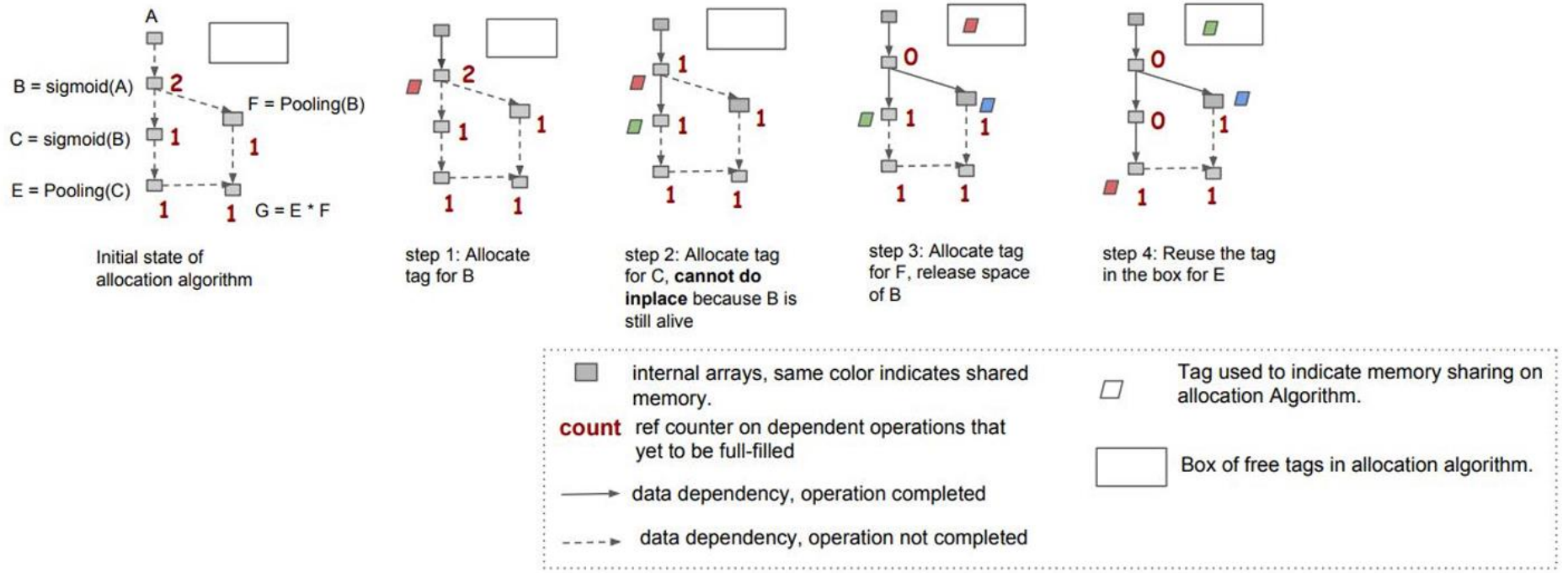
Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused



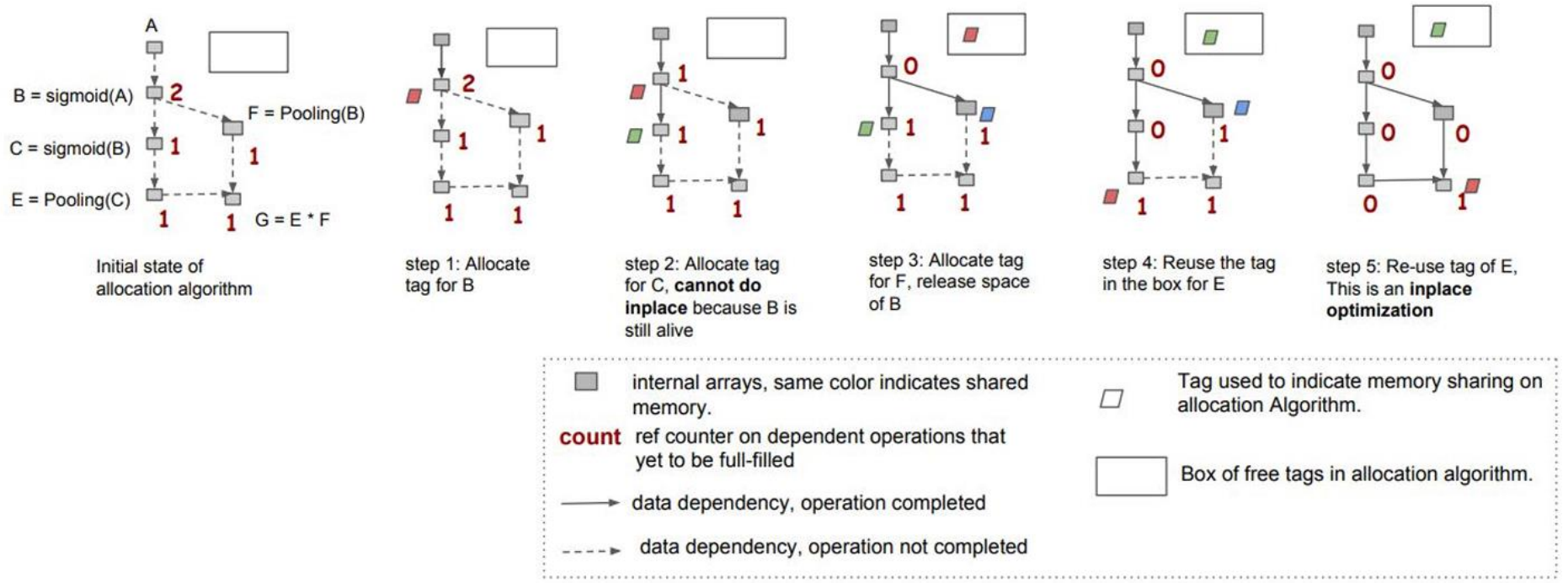
Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused



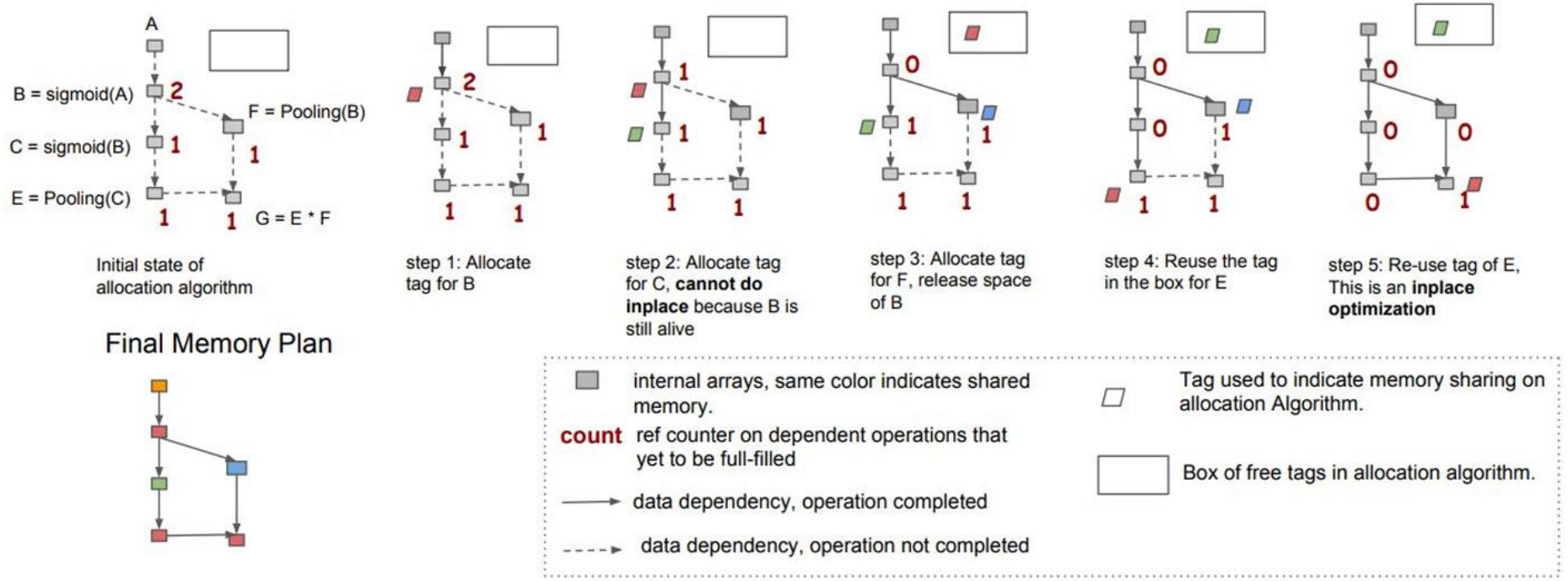
Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused



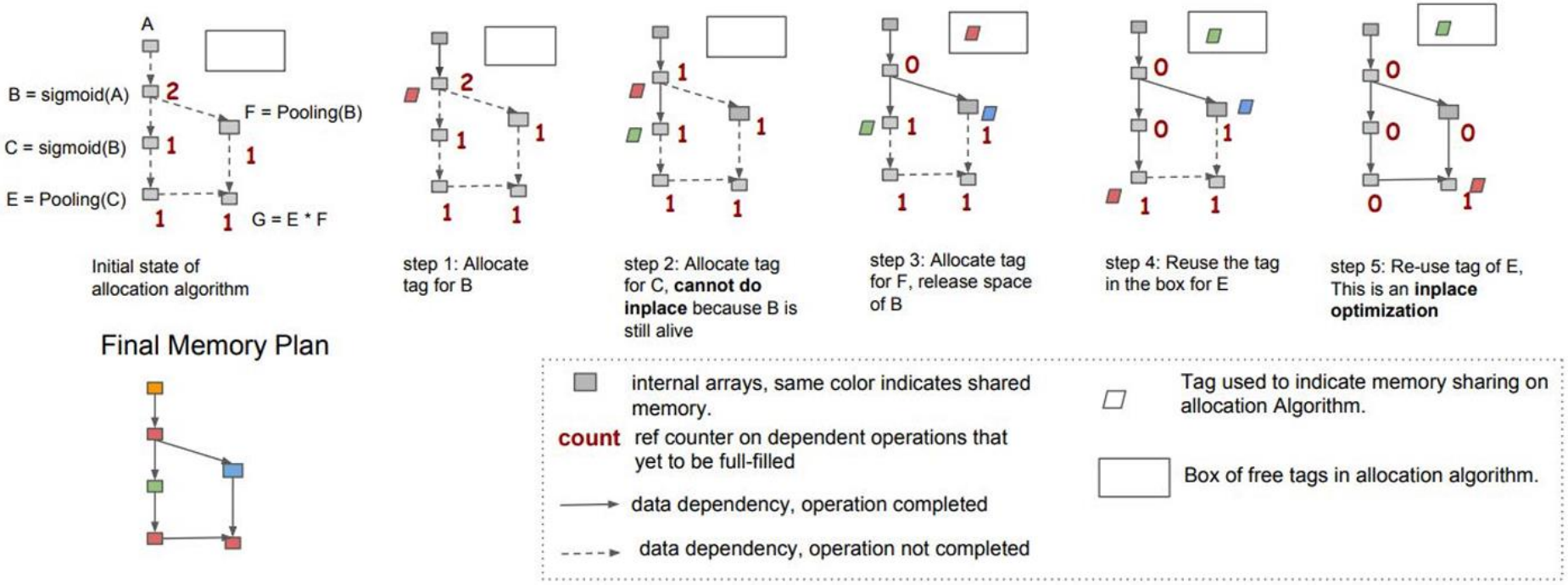
Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused



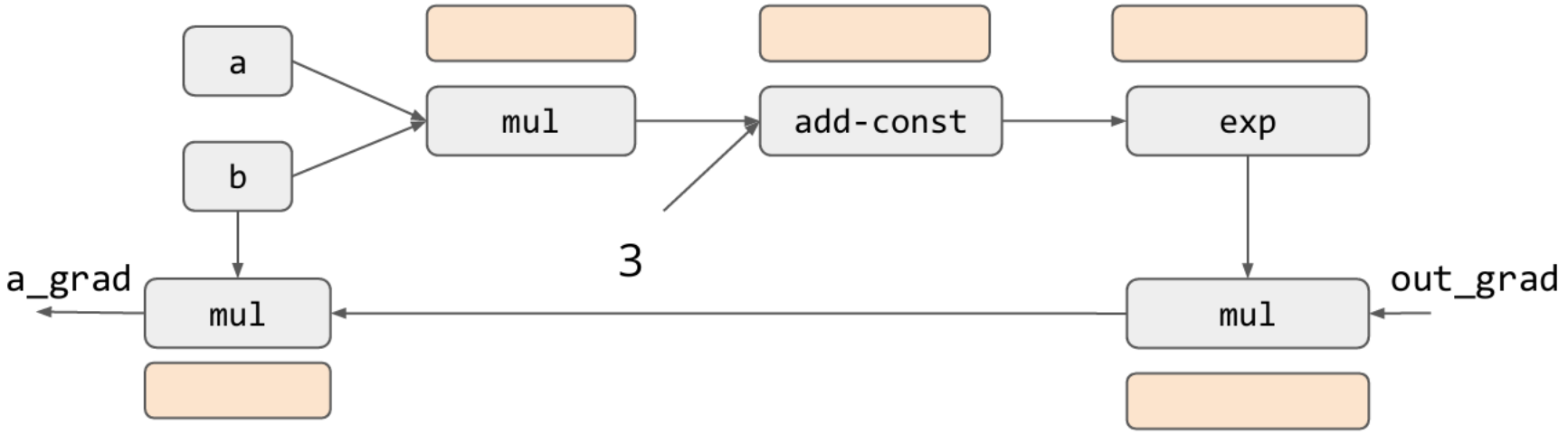
Memory Allocation/Planning Algorithm

- Static Memory Planning: plan for reuse ahead of time
- Liveness counters: Track the number of operations that need to be fulfilled before memory can be reused
- Dependencies: Important to declare minimum dependencies to optimize memory usage (DL frameworks already handle this for common operators)



Memory Plan with Gradient Calculation

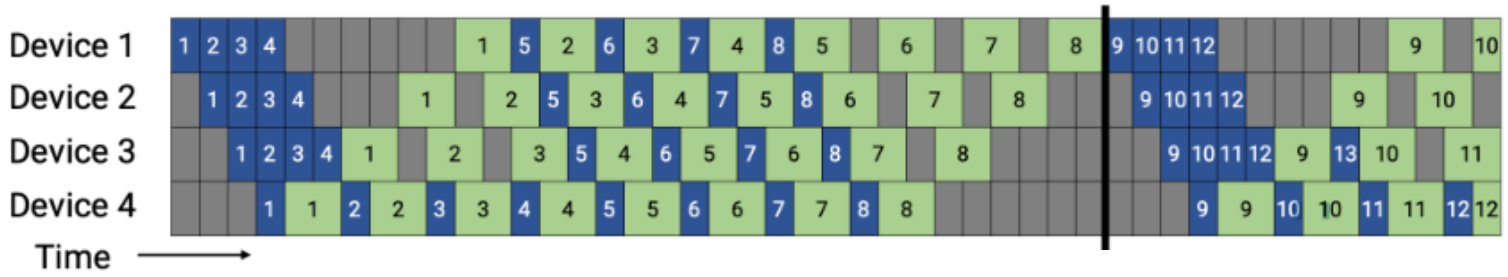
We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?



Activation Checkpointing

We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?

Idea: Drop some of the intermediate results and recover them from an extra forward computation (from the closest checkpointed results) when needed.



Activation Checkpointing

We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?

Idea: Drop some of the intermediate results and recover them from an extra forward computation (from the closest checkpointed results) when needed.

Algorithm:

1. Divide the network into several segments;
2. Store the output of each segment and drops all the intermediate results within each segment;

Activation Checkpointing

We can save memory usage by a **constant factor** using memory sharing for training, how can we do better?

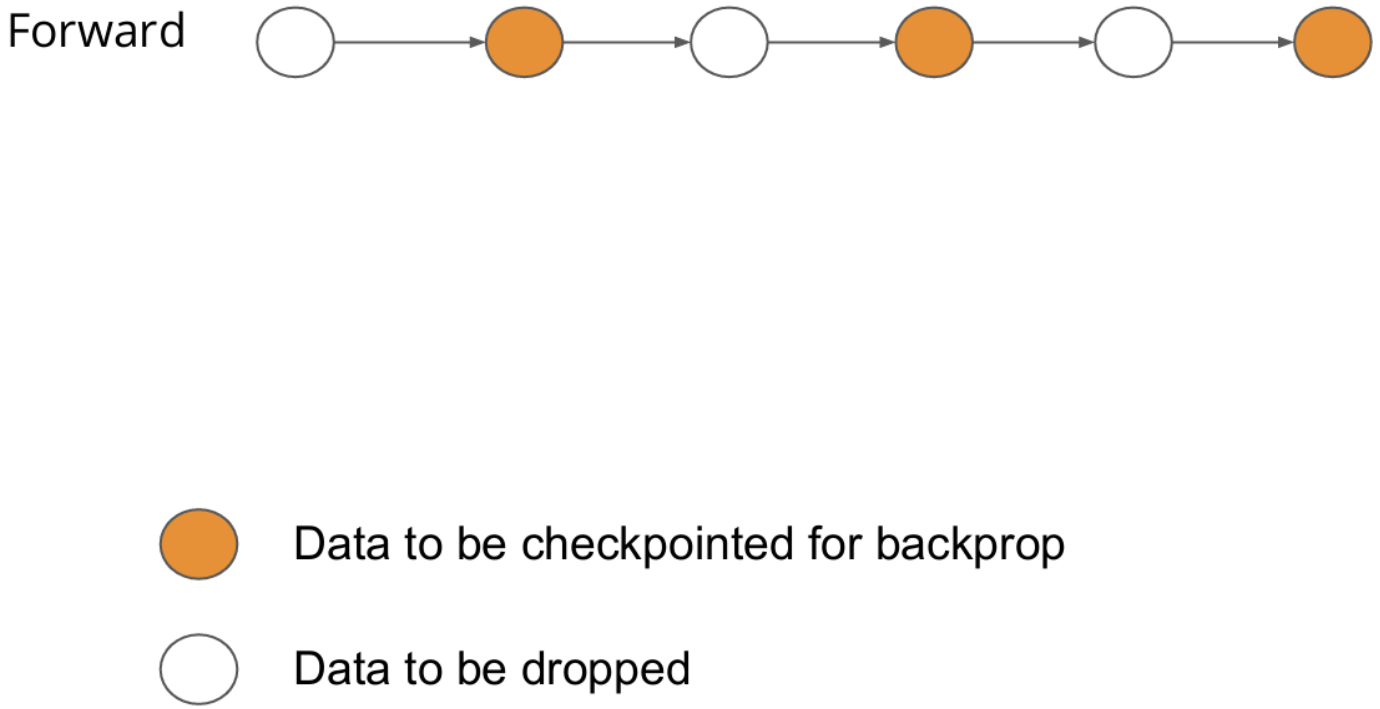
Idea: Drop some of the intermediate results and recover them from an extra forward computation (from the closest checkpointed results) when needed.

Algorithm:

1. Divide the network into several segments;
2. Store the output of each segment and drops all the intermediate results within each segment;
3. Recompute dropped results at segment-level during back-propagation

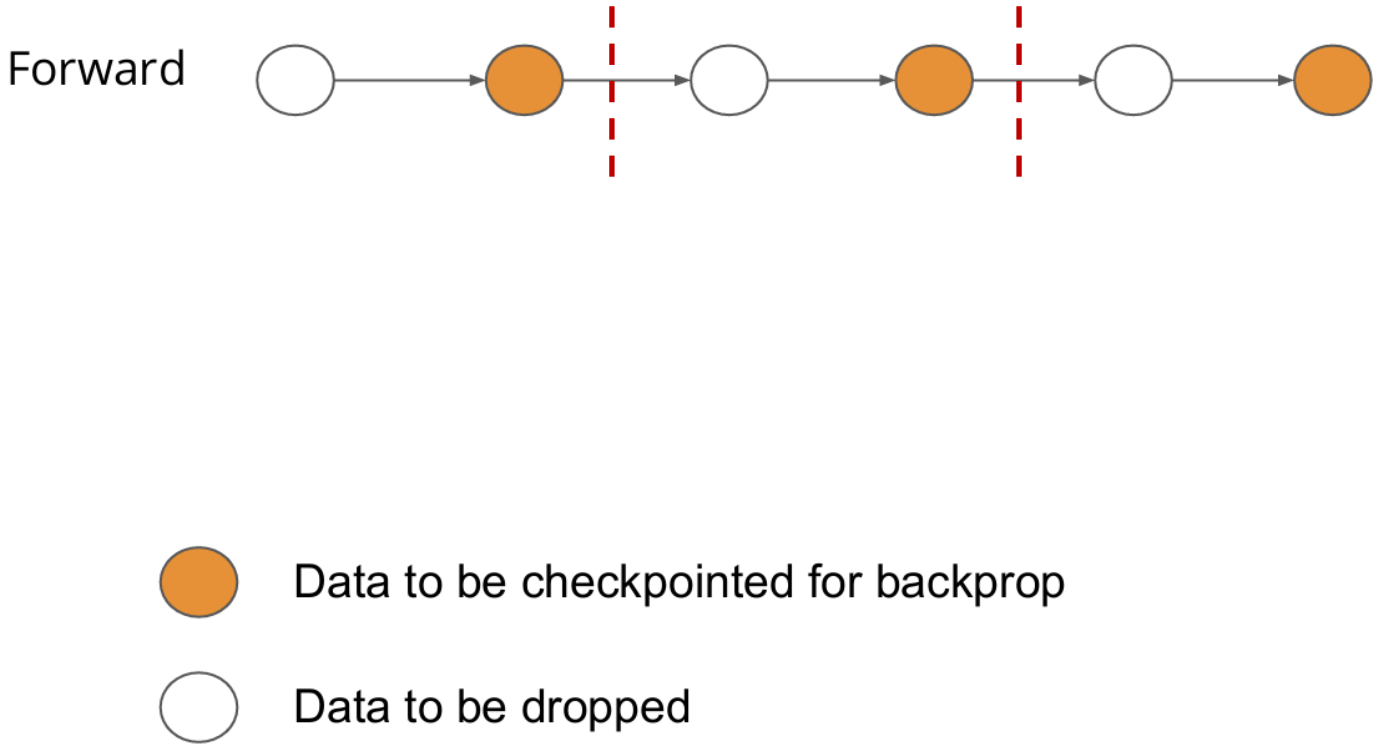
Activation Checkpointing

Checkpointing: storing only a few intermediate results during the forward pass.
Recomputation: Recomputing the necessary values during the backward for gradient calculation



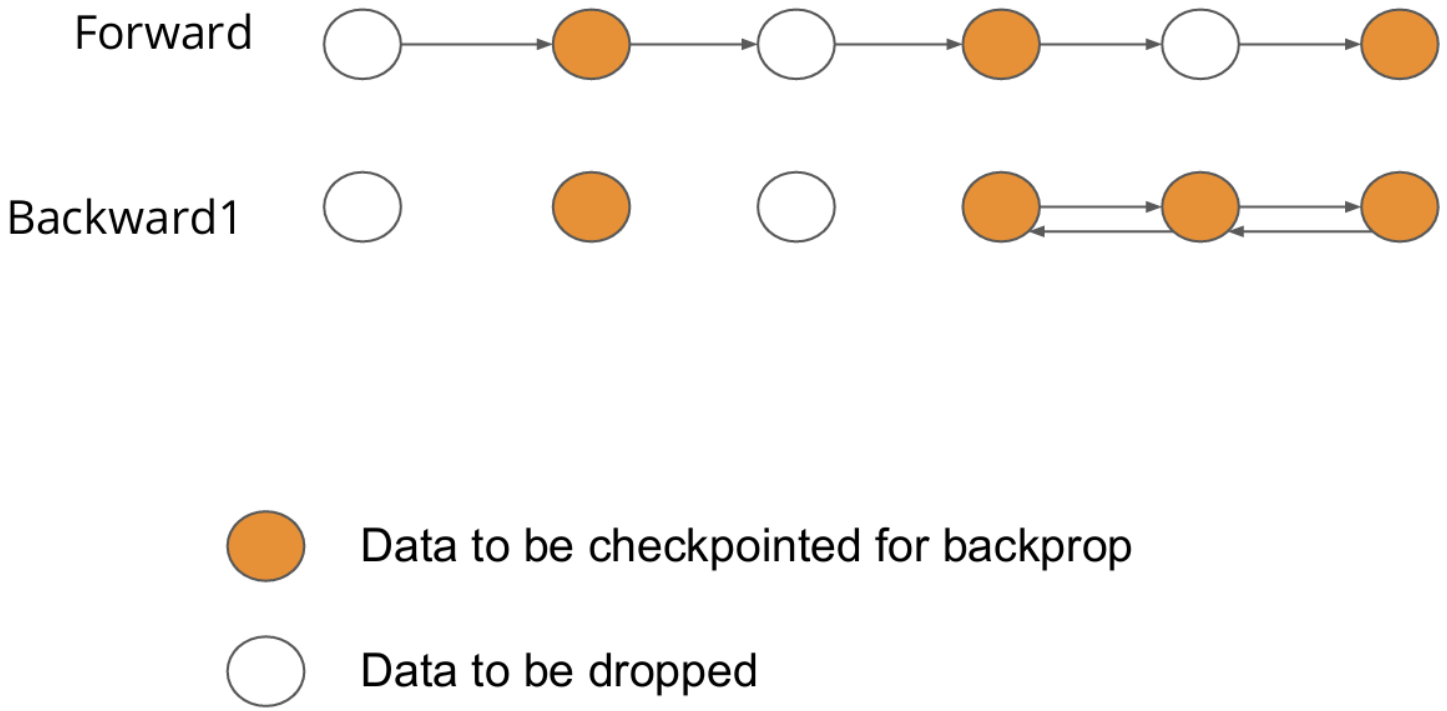
Activation Checkpointing

Checkpointing: storing only a few intermediate results during the forward pass.
Recomputation: Recomputing the necessary values during the backward for gradient calculation



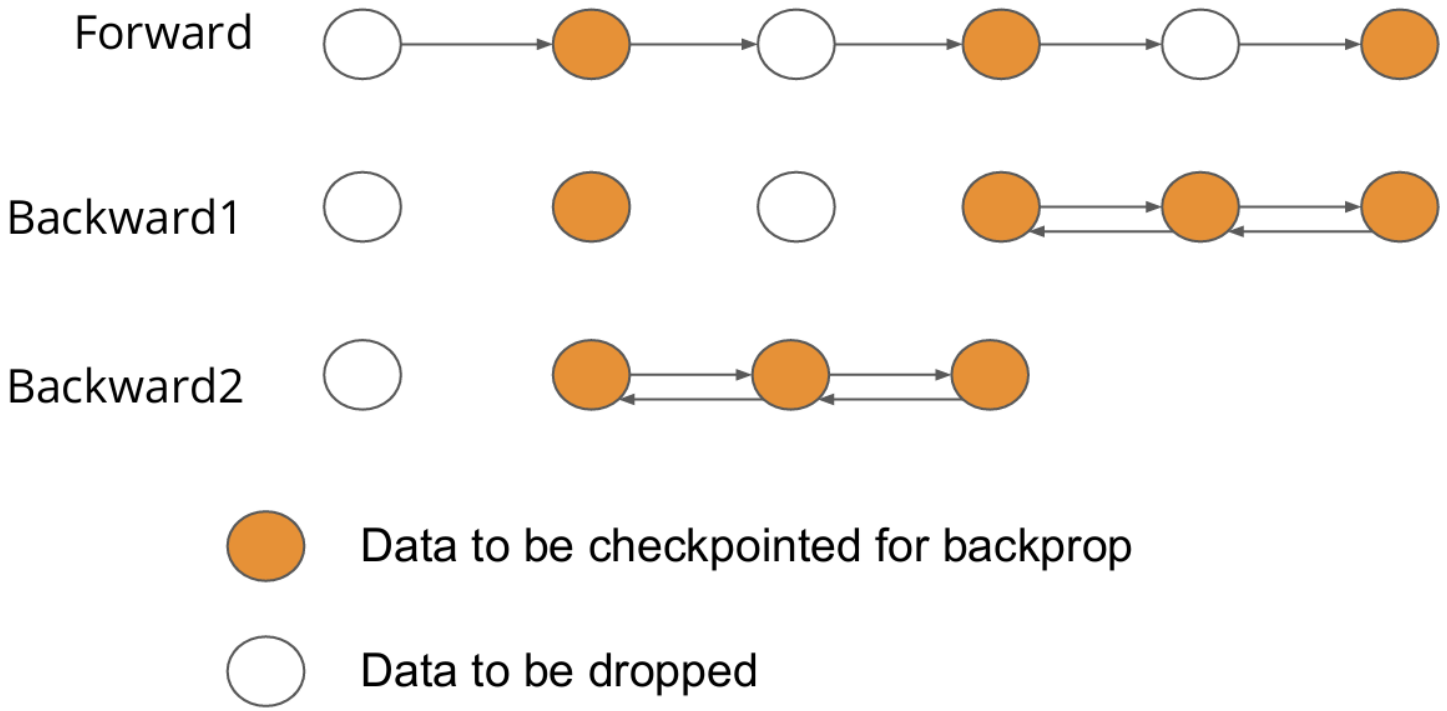
Activation Checkpointing

Checkpointing: storing only a few intermediate results during the forward pass.
Recomputation: Recomputing the necessary values during the backward for gradient calculation



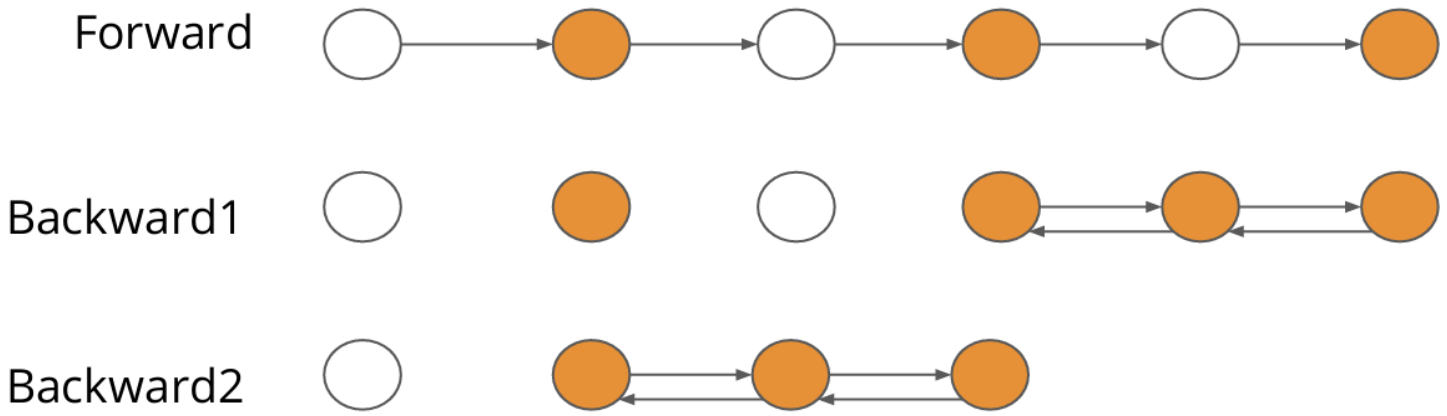
Activation Checkpointing

Checkpointing: storing only a few intermediate results during the forward pass.
Recomputation: Recomputing the necessary values during the backward for gradient calculation



Activation Checkpointing

Checkpointing: storing only a few intermediate results during the forward pass.
Recomputation: Recomputing the necessary values during the backward for gradient calculation



Selective Activation Checkpointing: Drop the results of low compute cost operations, e.g., Conv-BatchNorm-Activation, drop results of the batch norm, activation, and pooling, but keep the results of convolution

Analysis of Activation Checkpointing: Sublinear Memory Cost

Assume we divide the n network into k segments, the activation memory to train the network:

$$\text{cost-total} = \max_{i=1,\ldots,k} \text{cost-of-segment}(i) + O(k) = O\left(\frac{n}{k}\right) + O(k)$$

Max memory demand across segments

Analysis of Activation Checkpointing: Sublinear Memory Cost

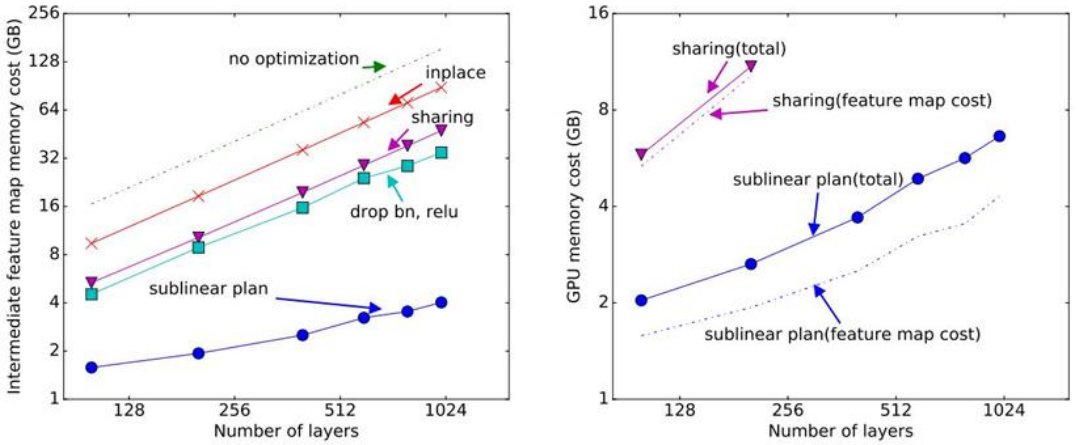
Assume we divide the n network into k segments, the activation memory to train the network:

$$\text{cost-total} = \max_{i=1,\ldots,k} \text{cost-of-segment}(i) + O(k) = O\left(\frac{n}{k}\right) + O(k)$$

Assume segments are equally divided, setting $k = \sqrt{n}$, we get $O(2\sqrt{n})$ with only one additional forward pass.

Results: Memory Cost

Running different schemes on ResNet



(a) Feature map memory cost estimation

(b) Runtime total memory cost

Training time roughly increases 30%.

Summary: Memory Optimization

- **Memory management**
  - Dynamic Memory Allocation: flexible, efficient but fragmentation
  - Static Memory Allocation: no runtime overhead but waste
  - In-place allocation and memory sharing (based on liveness)

- **Activation checkpointing**
  - Storing only a few intermediate results during the forward pass
  - Recomputing the necessary values during the backward for gradient calculation

Pannel Discussion

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

Samyam Rajbhandari*, Jeff Rasley*, Olatunji Ruwase, Yuxiong He
{samyamr, jerasley, olruwase, yuxhe}@microsoft.com

ZeRO partitions optimizer states and model weights across machines to reduce peak memory demands at the cost of higher communication.

Questions?